classroom(*building*, *room_number*, *capacity*)
department(*dept_name*, *building*, *budget*)
course(*course_id*, *title*, *dept_name*, *credits*)
instructor(*ID*, *name*, *dept_name*, *salary*)
section(*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)
teaches(*ID*, *course_id*, *sec_id*, *semester*, *year*)
student(*ID*, *name*, *dept_name*, *tot_cred*)
takes(*ID*, *course_id*, *sec_id*, *semester*, *year*, *grade*)
advisor(*s_ID*, *i_ID*)
time_slot(*time_slot_id*, *day*, *start_time*, *end_time*)
prereq(*course_id*, *prereq_id*)

Fig.0 大学数据库

1.使用 Fig.0 的大学模式（university schema）编写的 SQL 查询：

a) 查找每位至少选修过一门计算机科学课程的学生的 ID 和姓名，确保结果中没有重复的姓名。

```
SELECT DISTINCT student.ID, student.name
FROM student INNER JOIN takes  ON student.ID = takes.ID
             INNER JOIN course ON takes.course_id = course.course_id
WHERE course.dept_name = 'Comp. Sci.';
```

b) 查找每位未选修过 2017 年之前开设的任何课程的学生的 ID 和姓名。

```
SELECT ID, name
FROM student AS S
WHERE NOT EXISTS (
    SELECT *
    FROM takes AS T
    WHERE year < 2017 AND T.ID = S.ID
)
```

c) 对于每个系（department），找到该系教师的最高薪水。

```
SELECT dept_name, MAX(salary)
FROM instructor
GROUP BY dept_name
```

d) 找到在所有系（departments）中计算出的每个系最高薪水的最低值。

```
WITH maximum_salary_within_dept(dept_name, max_salary) AS (
    SELECT dept_name, MAX(salary)
    FROM instructor
    GROUP BY dept_name
)
SELECT MIN(max_salary)
FROM maximum_salary_within_dept
```

2.考虑以下查询：

```
with dept_total (dept_name, value) as
        (select dept_name, sum(salary)
        from instructor
        group by dept_name),
dept_total_avg(value) as
        (select avg(value)
        from dept_total)
select dept_name
from dept_total, dept_total_avg
where dept_total.value >= dept_total_avg.value;
```

请将此查询重新编写，而不使用 WITH 子句。

```sql
SELECT dept_name
FROM (
    SELECT dept_name, SUM(salary) AS total_salary
    FROM instructor
    GROUP BY dept_name
) AS dept_total,
(
    SELECT AVG(total_salary) AS avg_salary
    FROM (
        SELECT SUM(salary) AS total_salary
        FROM instructor
        GROUP BY dept_name
    ) AS subquery
) AS dept_total_avg
WHERE dept_total.total_salary >= dept_total_avg.avg_salary;
```

3.不使用"unique"构造，重写 where 语句：

```
where unique (select title from course)
```

有两种方法:

```sql
WHERE 1 >= ALL (
    SELECT COUNT(*)
    FROM course
    GROUP BY title
)
```

```
WHERE NOT EXISTS (
    SELECT *
    FROM course AS c1, course AS c2
    WHERE c1.course_id != c2.course_id AND c1.title = c2.title
)
```

---

branch(*branch_name*, *branch_city*, *assets*)
customer (*ID*, *customer_name*, *customer_street*, *customer_city*)
loan (*loan_number*, *branch_name*, *amount*)
borrower (*ID*, *loan_number*)
account (*account_number*, *branch_name*, *balance*)
depositor (*ID*, *account_number*)

Fig. 1 银行数据库

4.考虑 Fig.1 中的银行数据库，其中主键已经用下划线标记。构建以下针对这个关系数据库的 SQL 查询。

a) 查找银行的每位顾客的 ID，他们拥有账户但没有贷款。

```
(SELECT ID FROM depositor)
EXCEPT
(SELECT ID FROM borrower)
```

b) 查找与顾客'12345'同住在相同街道和城市的每位顾客的 ID。

```
SELECT F.ID
FROM customer AS F, customer AS S
WHERE F.customer_street = S.customer_street
    AND F.customer_city = S.customer_city
    AND S.customer_id = '12345';
```

c) 查找每个至少有一位在银行拥有账户并住在“Harrison”的顾客的分行的名称。

```
SELECT DISTINCT branch_name
FROM account, depositor, customer
WHERE customer.id = depositor.id
    AND depositor.account_number = account.account_number
    AND customer_city = 'Harrison'
```

5.考虑 Fig.1 中的银行数据库，其中主键已经用下划线标记。构建以下针对这个关系数据库的 SQL 查询。

a) 找到每位在“Brooklyn”的每个分行都有账户的顾客。

```
WITH all_branches_in_brooklyn(branch_name) AS (
    SELECT branch_name
    FROM branch
    WHERE branch_city = 'Brooklyn'
)
SELECT ID, customer_name
FROM customer AS c1
WHERE NOT EXISTS (
    (SELECT branch_name FROM all_branches_in_brooklyn)
    EXCEPT
    (
        SELECT branch_name
        FROM account INNER JOIN depositor
            ON account.account_number = depositor.account_number
        WHERE depositor.ID = c1.ID
    )
)
```

b）找到银行中所有贷款金额的总和。

```
SELECT SUM(amount)
FROM loan
```

c）找到具有资产超过至少一个位于“Brooklyn”的分行的资产的所有分行名称。

```
SELECT branch_name
FROM branch
WHERE assets > SOME (
    SELECT assets
    FROM branch
    WHERE branch_city = 'Brooklyn'
);
```